

Question one

- a) Search in AI refers to the systematic process of navigating through a state space starting from an initial state and applying valid operator rules in order to find a path to a desired goal state.

Informed Search – This strategy utilizes problem-specific knowledge to find the goal more efficiently. It evaluates which paths seem most promising and prioritizes them. It uses a heuristic function “ $h(n)$ ” to estimate the cost from the current node to the goal.

Uninformed Search - This strategy operates with absolutely no domain knowledge about the problem other than the initial state, the allowable actions, and the goal test. It explores the search space mechanically. It does not know if one state is better or closer to the goal than another.

- b) *import heapq*

def a_star_search(graph, start, goal, heuristics):

"""

A Search Algorithm implementation.*

graph: Adjacency list representing the graph with step costs.

start: The starting node.

goal: The target node.

heuristics: A dictionary mapping nodes to their estimated cost to the goal.

"""

Priority queue stores (f_cost, current_node, path)

open_set = []

heapq.heappush(open_set, (0 + heuristics[start], start, [start]))

Dictionary to keep track of the lowest g_cost to a specific node

g_costs = {start: 0}

while open_set:

Pop the node with the lowest f(n)

current_f, current_node, path = heapq.heappop(open_set)

If we reached the goal, return the optimized path and cost

if current_node == goal:

return path, g_costs[current_node]

Explore neighbors

for neighbor, step_cost in graph.get(current_node, {}).items():

Calculate exact cost from start to this neighbor

tentative_g_cost = g_costs[current_node] + step_cost

If this is a cheaper path to the neighbor, process it

if neighbor not in g_costs or tentative_g_cost < g_costs[neighbor]:

g_costs[neighbor] = tentative_g_cost

$f_cost = tentative_g_cost + heuristics[neighbor]$

Add the neighbor and its path to the priority queue

$new_path = path + [neighbor]$

$heapq.heappush(open_set, (f_cost, neighbor, new_path))$

return None, float('inf') # Return if no path exists

- c) i) **Valid** - A formula is valid if it evaluates to True under every possible assignment of truth values to its variables.

Satisfiable - A formula is satisfiable if there is at least one assignment of truth values that makes it true.

Unsatisfiable - A formula is unsatisfiable if it evaluates to False under every possible assignment of truth values.

Falsifiable: A formula is falsifiable if there is at least one assignment of truth values that makes it false

- ii) $P \wedge Q \rightarrow \neg R$

P	Q	R	$P \wedge Q$
T	T	T	T
T	T	F	T
T	F	T	F
F	F	F	F
F	T	T	F
F	T	F	F
F	F	T	F
F	F	F	F

Satisfiable – It is satisfiable as it is attained in any condition where P, Q and R are not all simultaneously true

Falsifiable – It is falsifiable as it is attained uniquely under the condition where P is true, Q is true and R is True

Valid – It is not valid because there is one instance where it evaluates to false

Unsatisfiable – It is not unsatisfiable because it frequently evaluates to true.

Question two

- a) **PEAS Framework**

Performance Measure – High accuracy/ match rate in positive identifications, speed of processing sample backlogs, low false-positive rates, maintenance of data privacy.

Environment - The forensic laboratory, DNA databases, physical tissue or bone samples from corpses, saliva or blood reference samples from relatives.

Actuators - Robotic arms for sample sorting and pipetting, centrifuges, screen displays for outputting match reports, automated alerts to the Ministry of Health and Ministry of Interior.

Sensors - Optical sequencing sensors, mass spectrometers, barcode or RFID scanners for sample identification, keyboard/database input streams.

PAGE Framework

Percepts - Chemical properties of the DNA, genetic markers, database queries, sample barcodes.

Actions - Extract DNA, sequence genome, cross-reference sequences with the relative database, flag high-probability matches.

Goal - Successfully, rapidly, and accurately identify the victims to bring closure to families.

Environment - The isolated forensic environment and digital genome servers.

b) *class VacuumAgent:*

```
def __init__(self):
```

```
# The environment consists of two locations, 'A' and 'B'.
```

```
# Both start in a 'Dirty' state.
```

```
self.environment = {'A': 'Dirty', 'B': 'Dirty'}
```

```
# The agent starts in location 'A'
```

```
self.location = 'A'
```

```
def sense(self):
```

```
# Sensor: Detects the current location and its status
```

```
return self.location, self.environment[self.location]
```

```
def act(self):
```

```
# Actuator: Determines the action based on the percept (Reflex Agent)
```

```
location, status = self.sense()
```

```
print(f"Agent is at Location {location}. Status: {status}")
```

```
if status == 'Dirty':
```

```
    print(f"Action: SUCK dirt at Location {location}.")
```

```
    self.environment[location] = 'Clean'
```

```
else:
```

```
    print(f"Location {location} is already Clean.")
```

```

# Move to the other location

if location == 'A':
    print("Action: Move RIGHT to Location B.\n")
    self.location = 'B'
elif location == 'B':
    print("Action: Move LEFT to Location A.\n")
    self.location = 'A'

def clean_environment(self):
    # Goal: Run the agent until both locations are clean
    print("--- Starting Vacuum Agent ---")
    # Run for a set number of steps to ensure both sides are checked
    for step in range(3):
        self.act()

    print("--- Environment is fully Cleaned ---")
    print("Final State:", self.environment)

# Initialize and run the agent
agent = VacuumAgent()
agent.clean_environment()

```